Backend Workflow: `app_upload-image_adam`

Overview

`app_upload-image_adam` is a Bubble.io backend API workflow that accepts a base64-encoded image, uploads it to private file storage, creates a Photo record linked to an asset, and returns the result. It is the server-side endpoint the Flutter app calls to upload inspection photos.

Trigger — API Event (`app_upload-image_Adam is called`)

Property	Value
Endpoint name	`app_upload-image_Adam`
Expose as public API workflow	On
Authentication	User & admin
Ignore privacy rules	Off
Trigger method	`POST`
Parameter definition	Manual
Response type	JSON Object
Return 200 if condition not met	Off

Parameters (both required, type `text`):

- `base64` — the base64-encoded image data
- `asset_id` — the ID of the asset to attach the photo to

Step 1 — EZ Uploader: Upload Private File [Paid Plan app]

Uses the **EZ Uploader** plugin to push the image into Bubble's private file storage.

Param	Value
(path) App host name	`flutterflowtest.bubbleapps.io/version-test`
(param) contents	`base64` *(the workflow's base64 input)*
(param) name	`blockimage.jpg` *(static filename)*
(param) attach_to	`asset_id` *(the workflow's asset_id input)*

> ***Note:*** *the filename is hardcoded to `blockimage.jpg`, so every uploaded file shares the same name (the storage system de-duplicates via its own path/key).*

Step 2 — Create a new Photo...

Creates a record in the **Photo** data type.

Field	Value
Image	`Result of step 1 (EZ Uploader – Upload…)'s body`

The uploaded file (returned in step 1's response body) is stored on the Photo's `Image` field. The Photo's link to the asset is established via step 1's `attach_to` → `asset_id`.

Step 3 — Return data from API

Returns the result of the workflow to the caller (the Flutter app) as a JSON Object.

Summary of the flow

```
POST { base64, asset_id }
        │
        ▼
Step 1: EZ Uploader → upload base64 as "blockimage.jpg" to private storage, attach_to asset_id
        │
        ▼
Step 2: Create Photo record, Image = step 1's body (uploaded file)
        │
        ▼
Step 3: Return data from API  (JSON Object)
```

Observations / potential issues

1. **Hardcoded filename** — `blockimage.jpg` is static. Fine if storage namespaces by record/key, but it is a code smell.
2. **Step 2 only sets `Image`** — there is no explicit `asset_id` (or relationship field) set on the Photo record itself in the visible config. The asset linkage relies on EZ Uploader's `attach_to`. If the app expects to query Photos *by asset*, confirm the Photo↔Asset relationship is actually persisted (you may want to add an `Asset` field on the Photo set to the `asset_id` input).
3. **`-adam` suffix** — naming suggests this is a developer test/experimental variant (compare with `app_upload-image`, `app_upload-image-direct`). Worth confirming which one is canonical before relying on it in production.